

Database Project Report
CS6083 Spring 2026
snickr: A Web-Based Collaboration System

Christine Wagner Hongyi Zeng
caw561@nyu.edu hz3866@nyu.edu

April 24, 2026

Contents

1	Introduction	2
2	Database Schema Design	2
2.1	ER Diagram	2
2.2	Relational Schema	2
2.3	Design Assumptions	3
2.4	Application-Level Enforcement	4
2.5	Space Efficiency	4
3	Schema Creation	5
4	Queries	6
4.1	(1) Create a new user account	6
4.2	(2) Create a new public channel (with authorization check)	6
4.3	(3) For each workspace, list all current administrators	6
4.4	(4) Pending invitations per public channel (older than 5 days)	7
4.5	(5) List all messages in a channel in chronological order	7
4.6	(6) List all messages posted by a particular user	8
4.7	(7) Search accessible messages by keyword	8
5	Test Data	9
5.1	Data Structure Overview	9
5.2	Sample Data Snapshot	9
6	System Architecture	10
6.1	Technology Stack	10
6.2	Architecture Diagram	10
6.3	RPC API Design	10
6.4	Available API Functions	10
7	Implementation Details	11
7.1	Backend Implementation	11
7.2	Frontend Implementation	11
7.3	Key Features	12

8	Security	13
8.1	SQL Injection Prevention	13
8.2	Cross-Site Scripting (XSS) Prevention	13
8.3	Cross-Site Request Forgery (CSRF)	13
8.4	Concurrency and Transaction Safety	13
9	User Guide	15
9.1	Getting Started	15
9.2	Main Interface	15
9.3	Top Bar Actions	15
9.4	Workspace Management	15
9.5	Channel Management	16
10	Session Logs	17
10.1	Session 1: Registration and Workspace Setup	17
10.2	Session 2: Invitations and Collaboration	17
10.3	Session 3: Admin Operations	18
11	Design Decisions and Revisions for Project 2	19
11.1	Schema Revisions	19
11.2	Design Decisions	19
11.3	Known Limitations	19
12	Conclusion	19

1 Introduction

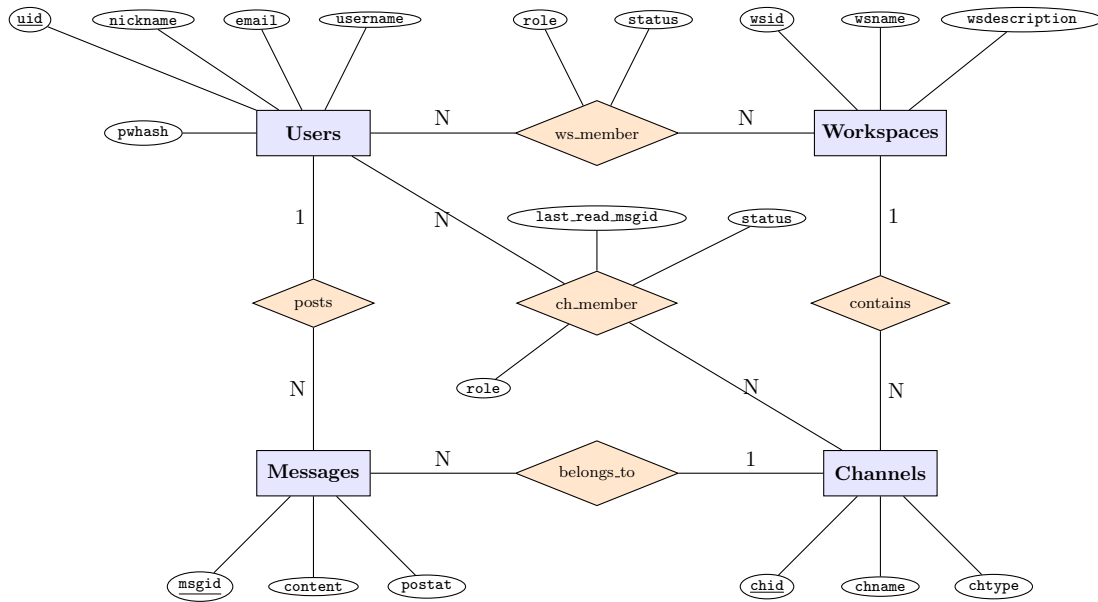
This report documents the design and implementation of *snickr*, a web-based collaboration system similar to Slack. The system allows users to create workspaces, organize communication through public, private, and direct channels, and exchange messages within those channels.

The database is built on PostgreSQL and consists of six relations: **users**, **workspaces**, **workspace_members**, **channels**, **channel_members**, and **messages**. Access control is enforced at the application level rather than through database permissions or views — a single DBMS account is used, and the application restricts content visibility based on workspace and channel membership.

This report covers both Project 1 (schema design and queries) and Project 2 (web interface implementation). Section 2 presents the database schema design with ER diagrams. Section 3 provides the schema creation SQL. Section 4 documents the seven required queries. Section 5 discusses the test data. Sections 6–10 cover the Project 2 web application: architecture, implementation details, security measures, and sample session logs.

2 Database Schema Design

2.1 ER Diagram



2.2 Relational Schema

The ER diagram translates into the following six relations:

1. **users**(uid, email, username, nickname, pwhash, createdat, updatedat)
 - email is UNIQUE NOT NULL
2. **workspaces**(wsid, wsname, wsdescription, createdat, updatedat)
3. **workspace_members**(wsid, uid, role, status, createdat, updatedat)
 - $wsid \rightarrow workspaces(wsid)$

- `uid` $\rightarrow$ `users(uid)`
 - `role` $\in$ `{creator, admin, member}`
 - `status` $\in$ `{pending, accepted, rejected}`
4. **channels**(`chid`, `wsid`, `chname`, `chtype`, `createdat`, `updatedat`)
- `wsid` $\rightarrow$ `workspaces(wsid)`
 - `chtype` $\in$ `{public, private, direct}`
5. **channel_members**(`chid`, `uid`, `role`, `status`, `last_read_msgid`, `createdat`, `updatedat`)
- `chid` $\rightarrow$ `channels(chid)`
 - `uid` $\rightarrow$ `users(uid)`
 - `role` $\in$ `{creator, member}`
 - `status` $\in$ `{pending, accepted, rejected}`
6. **messages**(`msgid`, `chid`, `uid`, `content`, `postat`)
- `chid` $\rightarrow$ `channels(chid)`
 - `uid` $\rightarrow$ `users(uid)`

2.3 Design Assumptions

- **A surrogate uid is used as the primary key for users**, rather than `email`. This allows users to change their email address without cascading updates across all foreign keys. The `email` column is constrained as `UNIQUE NOT NULL`.
- **Creator is stored in membership tables, not in workspace/channel tables**. The creator is the first member inserted with `role = 'creator'`. As specified in the requirements, the creator is treated as the initial administrator; query (3) therefore includes both `'creator'` and `'admin'` roles when listing administrators. Storing the creator as a role avoids redundancy and simplifies queries.
- **Invitations are tracked via the status field**, not as a separate entity. When a user is invited to a workspace or channel, a membership record is inserted with `status = 'pending'`. Upon acceptance, it is updated to `'accepted'`, and users may also explicitly decline an invitation by setting `'rejected'`. This design supports richer user interaction without introducing a separate invitations table.
- **Public channels are visible to all workspace members but still require explicit membership**. Users are not automatically added upon channel creation; instead, they must choose to join. When a user joins a public channel, a record is inserted into `channel_members` with `status = 'accepted'`, granting access to the channel's messages.
- **Passwords are stored as SHA-256 hashes**. The `pwhash` field stores a hashed password rather than plaintext for security.
- **Timestamps are recorded for all actions**. All tables include `createdat` and `updatedat` fields (messages use `postat` instead). While `updatedat` is not exercised by the required queries, it is included to support features such as profile edits, role changes, and invitation acceptance tracking.

- **Member removal** is supported by deleting the corresponding row from `workspace_members` or `channel_members`. No soft-delete flag is needed since the membership record itself represents the relationship.
- **Direct channels are limited to two members**, enforced at the application level rather than via a database constraint, since a `CHECK` on member count would require a trigger.
- **Message read state is tracked at the channel level**. Each `channel_members` record includes a `last_read_msgid` attribute, which stores the most recent message viewed by the user in that channel.
- **Access control is enforced at the application level**, not through database permissions or views, as specified in the project requirements.

2.4 Application-Level Enforcement

The following constraints are enforced by the web application rather than the database, either because SQL `CHECK` constraints cannot express them without triggers, or because they involve session state:

- **Direct channel membership limit**. The application ensures that exactly two users are associated with a direct channel at creation time.
- **Workspace invitation authority**. Only users with `role` IN ('creator', 'admin') in `workspace_members` are permitted to insert new membership records for other users.
- **Private channel invitation authority**. Only the channel creator (`role` = 'creator' in `channel_members`) may invite additional users to a private channel.
- **Session identification**. Each HTTP request includes a session cookie identifying the logged-in user `uid`. The application resolves this identity before executing any query.
- **Public channel join eligibility**. Before inserting a `channel_members` row for a public channel, the application verifies that the user has `status` = 'accepted' in the corresponding `workspace_members` relation.

2.5 Space Efficiency

- **No redundant creator columns**. Instead of storing a separate `creator_uid` in the `workspaces` and `channels` tables, we encode the creator as a membership record with `role` = 'creator'.
- **Composite primary keys** on `workspace_members` and `channel_members` eliminate the need for surrogate ID columns in junction tables.
- **VARCHAR with appropriate limits** is used instead of fixed-length `CHAR` fields, so shorter values do not waste space.
- **SERIAL auto-increment** is used for `uid`, `wsid`, `chid`, and `msgid` (4-byte integers), which is more compact than UUIDs.
- **No separate invitations table**. The `status` field on membership tables tracks invitation state in-place, avoiding an extra table and the joins it would require.

3 Schema Creation

The following SQL creates the schema in PostgreSQL with all key, foreign key, and check constraints:

```
CREATE TABLE users (
    uid          SERIAL          PRIMARY KEY,
    email        VARCHAR(255)    NOT NULL UNIQUE,
    username     VARCHAR(50)     NOT NULL,
    nickname     VARCHAR(50),
    pwhash       VARCHAR(255)    NOT NULL,
    createdat    TIMESTAMP       NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updatedat    TIMESTAMP       NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE workspaces (
    wsid         SERIAL          PRIMARY KEY,
    wsname       VARCHAR(100)    NOT NULL,
    wsdescription TEXT,
    createdat    TIMESTAMP       NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updatedat    TIMESTAMP       NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE workspace_members (
    wsid         INT             NOT NULL REFERENCES workspaces(wsid),
    uid          INT             NOT NULL REFERENCES users(uid),
    role         VARCHAR(10)     NOT NULL CHECK (role IN ('creator', 'admin', 'member')),
    status       VARCHAR(10)     NOT NULL CHECK (status IN ('pending', 'accepted', 'rejected'))
                                DEFAULT 'pending',
    createdat    TIMESTAMP       NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updatedat    TIMESTAMP       NOT NULL DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (wsid, uid)
);

CREATE TABLE channels (
    chid         SERIAL          PRIMARY KEY,
    wsid         INT             NOT NULL REFERENCES workspaces(wsid),
    chname       VARCHAR(100),
    chtype       VARCHAR(10)     NOT NULL CHECK (chtype IN ('public', 'private', 'direct')),
    createdat    TIMESTAMP       NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updatedat    TIMESTAMP       NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE channel_members (
    chid         INT             NOT NULL REFERENCES channels(chid),
    uid          INT             NOT NULL REFERENCES users(uid),
    role         VARCHAR(10)     NOT NULL CHECK (role IN ('creator', 'member')),
    status       VARCHAR(10)     NOT NULL CHECK (status IN ('pending', 'accepted', 'rejected'))
                                DEFAULT 'pending',
    last_read_msgid INT,
    createdat    TIMESTAMP       NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updatedat    TIMESTAMP       NOT NULL DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (chid, uid)
);

CREATE TABLE messages (
    msgid        SERIAL          PRIMARY KEY,
    chid         INT             NOT NULL REFERENCES channels(chid),
    uid          INT             NOT NULL REFERENCES users(uid),
    content      TEXT            NOT NULL,
    postat       TIMESTAMP       NOT NULL DEFAULT CURRENT_TIMESTAMP
);
```

4 Queries

4.1 (1) Create a new user account

```
CREATE OR REPLACE PROCEDURE create_user(  
    p_email    VARCHAR,  
    p_username VARCHAR,  
    p_nickname VARCHAR,  
    p_pwhash   VARCHAR  
)  
LANGUAGE SQL  
AS $$  
    INSERT INTO users (email, username, nickname, pwhash)  
    VALUES (p_email, p_username, p_nickname, p_pwhash);  
$$;  
  
-- Example call:  
CALL create_user('gz1234@nyu.edu', 'gracezhu', 'Grace', 'a1b2c3d4e5f6');
```

Test: Calling with a new email succeeds and inserts a row into `users`. Calling with a duplicate email raises a unique constraint violation.

4.2 (2) Create a new public channel (with authorization check)

```
CREATE OR REPLACE PROCEDURE create_public_channel(  
    p_wsuid    INT,  
    p_uid      INT,  
    p_chname   VARCHAR  
)  
LANGUAGE plpgsql  
AS $$  
DECLARE  
    v_chid INT;  
BEGIN  
    IF NOT EXISTS (  
        SELECT 1 FROM workspace_members  
        WHERE wsid = p_wsuid AND uid = p_uid AND status = 'accepted'  
    ) THEN  
        RAISE EXCEPTION 'User % is not an accepted member of workspace %',  
            p_uid, p_wsuid;  
    END IF;  
    INSERT INTO channels (wsid, chname, chtype)  
    VALUES (p_wsuid, p_chname, 'public')  
    RETURNING chid INTO v_chid;  
    INSERT INTO channel_members (chid, uid, role, status)  
    VALUES (v_chid, p_uid, 'creator', 'accepted');  
END;  
$$;  
  
-- Example call:  
CALL create_public_channel(1, 1, 'announcements');
```

Test: Calling with an accepted workspace member succeeds; calling with a non-member raises an exception.

4.3 (3) For each workspace, list all current administrators

```
SELECT w.wsid, w.wsname, u.email, u.username, wm.role  
FROM workspaces w  
JOIN workspace_members wm ON w.wsid = wm.wsid  
JOIN users u ON wm.uid = u.uid  
WHERE wm.role IN ('creator', 'admin') AND wm.status = 'accepted'  
ORDER BY w.wsid, wm.role;
```

Result:

wsid	wsname	email	username	role
1	CS Department	bs5678@nyu.edu	bobsmith	admin
1	CS Department	az1234@nyu.edu	alicezhang	creator
2	Tandon Robotics Club	cw9012@nyu.edu	carolwang	admin
2	Tandon Robotics Club	bs5678@nyu.edu	bobsmith	creator
3	DB Project Team	gs6789@nyu.edu	gracesun	admin
3	DB Project Team	cw9012@nyu.edu	carolwang	creator

4.4 (4) Pending invitations per public channel (older than 5 days)

```
SELECT c.chid, c.chname, COUNT(cm.uid) AS pending_count
FROM channels c
LEFT JOIN channel_members cm ON c.chid = cm.chid
    AND cm.status = 'pending'
    AND cm.createdat < CURRENT_TIMESTAMP - INTERVAL '5 days'
WHERE c.wsid = 1 AND c.chtype = 'public'
GROUP BY c.chid, c.chname
ORDER BY c.chid;
```

Result:

chid	chname	pending_count
1	general	1
4	research	0

4.5 (5) List all messages in a channel in chronological order

```
CREATE OR REPLACE FUNCTION list_channel_messages(p_chid INT)
RETURNS TABLE (
    msgid INT, email VARCHAR, username VARCHAR,
    content TEXT, postat TIMESTAMP
)
LANGUAGE SQL
AS $$
    SELECT m.msgid, u.email, u.username, m.content, m.postat
    FROM messages m
    JOIN users u ON m.uid = u.uid
    WHERE m.chid = p_chid
    ORDER BY m.postat ASC;
$$;

-- Example call:
SELECT * FROM list_channel_messages(1);
```

Result (channel 1, #general):

msgid	email	username	content	postat
1	az1234@nyu.edu	alicezhang	Welcome to the CS Department workspace!	2026-04-01 09:00
2	bs5678@nyu.edu	bobsmith	Thanks Alice! Excited to be here.	2026-04-01 09:05
3	cw9012@nyu.edu	carolwang	Is the new curriculum perpendicular...	2026-04-01 09:10
4	az1234@nyu.edu	alicezhang	Not exactly perpendicular, but quite different.	2026-04-01 09:15
5	gs6789@nyu.edu	gracesun	Hi everyone! Just joined the department.	2026-04-01 09:20

4.6 (6) List all messages posted by a particular user

```
SELECT m.msgid, c.chname, m.content, m.postat
FROM messages m
JOIN channels c ON m.chid = c.chid
WHERE m.uid = 1 -- Alice
ORDER BY m.postat DESC;
```

Result (Alice, uid=1): Returns all messages Alice has posted across #general, #hiring-committee, DM with Bob, and #research, ordered most recent first.

4.7 (7) Search accessible messages by keyword

```
CREATE OR REPLACE FUNCTION search_accessible_messages(
    p_uid INT, p_keyword VARCHAR
)
RETURNS TABLE (
    msgid INT, chname VARCHAR, author VARCHAR,
    content TEXT, postat TIMESTAMP
) LANGUAGE SQL AS $$
    SELECT m.msgid, c.chname, u.email AS author,
           m.content, m.postat
    FROM messages m
    JOIN channels c ON m.chid = c.chid
    JOIN users u ON m.uid = u.uid
    JOIN channel_members cm ON c.chid = cm.chid AND cm.uid = p_uid
    JOIN workspace_members wm
        ON c.wsid = wm.wsid AND wm.uid = p_uid
    WHERE cm.status = 'accepted'
        AND wm.status = 'accepted'
        AND m.content ILIKE '%' || p_keyword || '%'
    ORDER BY m.postat DESC;
$$;
-- Example call:
SELECT * FROM search_accessible_messages(1, 'perpendicular');
```

Result:

msgid	chname	author	content	postat
4	general	az1234@nyu.edu	Not exactly perpendicular, but quite different.	2026-04-01 09:15
3	general	cw9012@nyu.edu	Is the new curriculum perpendicular...	2026-04-01 09:10

Access control verification:

1. **Alice (uid=1):** Accepted in WS1 and channel #general $\Rightarrow$ 2 results (plus 1 from #research).
✓
2. **Dave (uid=4):** Pending in channel #general but accepted in WS2 #events $\Rightarrow$ only the #events match returned. ✓
3. **Eve (uid=5):** Pending workspace invitation $\Rightarrow$ 0 results. ✓

5 Test Data

The test dataset consists of **8 users**, **3 workspaces**, **11 channels** (covering public, private, and direct types), and **over 60 messages**.

5.1 Data Structure Overview

- **8 users** span three workspaces with overlapping membership. Bob, Carol, and Dave appear in multiple workspaces, testing cross-workspace queries.
- **3 workspaces:** CS Department (academic), Tandon Robotics Club (extracurricular), DB Project Team (course project). This variety creates realistic usage patterns.
- **Role diversity:** Alice is a workspace creator (WS1) and channel creator. Bob is both a workspace creator (WS2) and an admin (WS1). Carol is a workspace creator (WS3) and admin (WS2). This tests query (3).
- **Pending invitations:** Eve has a pending workspace invite; Dave and Frank have pending channel invites older than 5 days. This tests query (4).
- **60+ messages** are distributed across all channels with natural conversation patterns. Multiple messages contain the keyword “perpendicular” across different workspaces, testing query (7)’s keyword search with access control.
- **Diverse channel types** per workspace: each workspace has public channels, and at least one private channel or direct message channel.

5.2 Sample Data Snapshot

Users:

uid	email	username	nickname
1	az1234@nyu.edu	alicezhang	Alice
2	bs5678@nyu.edu	bobsmith	Bob
3	cw9012@nyu.edu	carolwang	Carol
4	dj3456@nyu.edu	davejohnson	Dave
5	ec7890@nyu.edu	evechen	Eve
6	fl2345@nyu.edu	frankli	Frank
7	gs6789@nyu.edu	gracesun	Grace
8	hk1234@nyu.edu	henrykim	Henry

Workspace Members (excerpt):

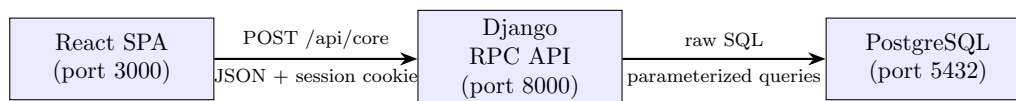
wsid	uid	role	status	notes
1	1	creator	accepted	Alice created CS Dept
1	2	admin	accepted	Bob is admin
1	5	member	pending	Eve invited, not joined
2	2	creator	accepted	Bob created Robotics
2	1	member	pending	Alice invited, not joined
3	3	creator	accepted	Carol created DB Project
3	7	admin	accepted	Grace is admin

6 System Architecture

6.1 Technology Stack

- **Backend:** Django 5.2 (Python) — serves the API via a single RPC endpoint
- **Database:** PostgreSQL 15 — all queries use raw SQL (no ORM)
- **Frontend:** React 18 with Tailwind CSS — single-page application
- **Deployment:** Docker Compose — PostgreSQL container + Django container

6.2 Architecture Diagram



6.3 RPC API Design

Rather than a RESTful API with multiple endpoints and HTTP verbs, we use a single **Remote Procedure Call (RPC)** endpoint at `POST /api/core`. Every request contains a JSON body:

```
{"function": "function_name", "args": [], "kwargs": {}}
```

The server dispatches to the named Python function, which executes the corresponding SQL queries and returns the result as JSON. This approach was chosen for simplicity — the backend is a single `views.py` file with a function registry, making it easy to add new operations without creating new URL patterns or view classes.

6.4 Available API Functions

Category	Functions
Auth	<code>register</code> , <code>login</code> , <code>logout</code> , <code>me</code>
Workspaces	<code>get_workspaces</code> , <code>create_workspace</code> , <code>delete_workspace</code> <code>get_workspace_members</code> , <code>invite_to_workspace</code> <code>respond_workspace_invite</code> , <code>update_workspace_member</code> , <code>leave_workspace</code>
Channels	<code>get_channels</code> , <code>create_channel</code> , <code>delete_channel</code> <code>get_channel_members</code> , <code>invite_to_channel</code> , <code>respond_channel_invite</code> <code>join_channel</code> , <code>leave_channel</code> , <code>get_public_channels</code>
Messages	<code>get_messages</code> , <code>send_message</code> , <code>search_messages</code> , <code>get_user_messages</code>
Users	<code>search_users</code>
Admin	<code>initialize</code>

7 Implementation Details

7.1 Backend Implementation

The entire backend is contained in `DBProject/views.py`. Key implementation choices:

- **Raw SQL only.** Every database operation uses `connection.cursor()` and parameterized SQL queries. Django's ORM is not used, in compliance with the course requirement. For example, the login function:

```
def login(request, username, password):
    with connection.cursor() as c:
        c.execute(
            "SELECT uid, email, username, nickname, pwhash "
            "FROM users WHERE username = %s",
            [username],
        )
        user = _dictfetchone(c)
    if not user or user["pwhash"] != _hash(password):
        return {"error": "invalid credentials"}
    request.session["uid"] = user["uid"]
    return user
```

- **Session-based authentication.** Django's built-in session framework stores the user's uid in a server-side session. A session cookie (`sessionid`) is sent to the browser. Every protected function checks `request.session.get("uid")` before proceeding.
- **Password hashing.** Passwords are hashed with SHA-256 before storage. The hash comparison is performed in Python, not in SQL.
- **Authorization checks.** Operations like `invite_to_workspace` first query the caller's role and verify it is 'creator' or 'admin' before proceeding.

7.2 Frontend Implementation

The frontend is a React single-page application with the following component hierarchy:

- `App.js` — top-level routing (login vs. chat page)
- `LoginPage.js` / `RegisterPage.js` — authentication forms
- `ChatPage.js` — main layout with state management
 - `WorkspaceList.js` — left sidebar listing workspaces
 - `ChannelList.js` — second sidebar listing channels
 - `ChatWindow.js` — message display and input area
 - `Inbox.js` — dropdown for pending invitations
 - `SearchModal.js` — keyword search across all accessible messages
 - Various modals: `CreateWorkspaceModal`, `CreateChannelModal`, `WorkspaceMembersModal`, `ChannelSettingsModal`, `DiscoverChannelsModal`

All API calls go through `services/api.js`, which wraps the RPC protocol and includes `credentials`: "include" for session cookie handling across the cross-origin boundary (React on port 3000, Django on port 8000).

7.3 Key Features

1. **User registration and login** — users create accounts with email, username, nickname, and password. Sessions persist across page refreshes via the `me` endpoint.
2. **Workspace management** — create workspaces, invite members (by searching usernames), promote/demote admins, remove members, leave or delete workspaces.
3. **Channel types** — public (discoverable, self-join), private (invite-only), and direct messages (two-person).
4. **Messaging** — send and view messages in real-time within a channel. Messages display chronologically with sender name and timestamp.
5. **Invitation inbox** — pending workspace and channel invitations appear in a dropdown; users can accept or reject each.
6. **Public channel discovery** — a “Browse” button lists public channels in the current workspace that the user has not yet joined. One-click join.
7. **Message search** — a search modal allows users to search for messages by keyword across all channels they have access to. Results show the channel name, author, content, and timestamp.

8 Security

8.1 SQL Injection Prevention

All database queries use **parameterized queries** with `%s` placeholders. User-supplied values are never concatenated into SQL strings. Django's database adapter passes parameters separately to PostgreSQL, which treats them as data, not executable SQL.

```
# Safe: parameterized query
c.execute(
    "SELECT uid, email, username, nickname FROM users "
    "WHERE username ILIKE %s OR email ILIKE %s LIMIT 20",
    [f"%{query}%", f"%{query}%"],
)
```

The stored procedures defined in `query.sql` (e.g., `create_user`, `create_public_channel`) also use parameterized inputs, providing an additional layer of protection when called directly.

8.2 Cross-Site Scripting (XSS) Prevention

The frontend uses React, which **automatically escapes all content rendered in JSX**. When a variable is placed inside curly braces (`{msg.content}`), React converts special characters (`<`, `>`, `&`, `"`, `'`) to their HTML entity equivalents before inserting them into the DOM. This prevents any user-supplied content (message text, usernames, channel names) from being interpreted as HTML or JavaScript.

We do not use `dangerouslySetInnerHTML` anywhere in the codebase, so there is no bypass of React's built-in XSS protection.

8.3 Cross-Site Request Forgery (CSRF)

The RPC endpoint uses `@csrf_exempt` because the frontend is a separate React application that communicates via JSON POST requests with session cookies. CSRF protection is less critical here because:

1. The endpoint only accepts POST with a `Content-Type: application/json` header.
2. Browsers enforce CORS preflight checks for cross-origin JSON requests, preventing third-party sites from forging requests with the correct content type.
3. Django's `CORS_ALLOWED_ORIGINS` is configured to only allow the frontend's origin.

8.4 Concurrency and Transaction Safety

Multi-step database operations are wrapped in `transaction.atomic()` to ensure atomicity. If any step fails, the entire transaction is rolled back, preventing partial state changes.

The following operations use explicit transactions:

- `create_workspace` — inserts into `workspaces` and `workspace_members`
- `create_channel` — checks membership, inserts into `channels` and `channel_members`
- `delete_channel` — deletes from `messages`, `channel_members`, and `channels`
- `delete_workspace` — cascading delete across five tables

- `invite_to_workspace` — checks role, then inserts invitation
- `update_workspace_member` — checks role, then modifies/removes member
- `initialize` — drops and recreates all tables with test data

Example:

```
def delete_workspace(request, wsid):
    uid = request.session.get("uid")
    if not uid:
        return {"error": "not logged in"}
    with transaction.atomic():
        with connection.cursor() as c:
            c.execute("SELECT role FROM workspace_members "
                      "WHERE wsid=%s AND uid=%s", [wsid, uid])
            row = _dictfetchone(c)
            if not row or row["role"] != "creator":
                return {"error": "only the workspace creator can delete it"}
            c.execute("DELETE FROM messages WHERE chid IN "
                      "(SELECT chid FROM channels WHERE wsid=%s)", [wsid])
            c.execute("DELETE FROM channel_members WHERE chid IN "
                      "(SELECT chid FROM channels WHERE wsid=%s)", [wsid])
            c.execute("DELETE FROM channels WHERE wsid=%s", [wsid])
            c.execute("DELETE FROM workspace_members WHERE wsid=%s", [wsid])
            c.execute("DELETE FROM workspaces WHERE wsid=%s", [wsid])
    return {"ok": True}
```

PostgreSQL's default isolation level (`READ COMMITTED`) ensures that concurrent sessions do not see uncommitted changes from other transactions. This prevents issues such as:

- A user reading messages from a channel that is being deleted by another user.
- Two users simultaneously creating a workspace and both getting the same `wsid`.
- An invitation being accepted while the workspace is being deleted.

Single-statement operations (e.g., `send_message`, `get_messages`) run in Django's default auto-commit mode, which is safe because a single SQL statement is inherently atomic in PostgreSQL.

9 User Guide

9.1 Getting Started

1. **Start the system:** Run `docker compose up --build` from the project root.
2. **Initialize the database:** Open `http://localhost:8000` and click the **Initialize** button to create tables and load test data.
3. **Open the frontend:** Navigate to `http://localhost:3000`.
4. **Log in or register:** Use a test account (e.g., `alicezhang` / `alice123`) or create a new account.

9.2 Main Interface

The main interface consists of three panels:

- **Left sidebar (purple):** Lists your workspaces. Click to select. The “+” button creates a new workspace.
- **Middle sidebar (gray):** Lists channels in the selected workspace. Click to open. Includes buttons for creating channels and browsing public channels.
- **Main area (white):** Displays messages in the selected channel. Type a message and click Send or press Enter.

9.3 Top Bar Actions

- **Search:** Opens a modal where you can search for messages by keyword across all your accessible channels. Results show the channel name, author, content, and timestamp.
- **Inbox:** Shows pending workspace and channel invitations. Accept or reject each with a single click.
- **Logout:** Ends the session and returns to the login page.

9.4 Workspace Management

Click “Members & settings” at the bottom of the workspace sidebar to:

- View all workspace members and their roles
- Invite new members by searching usernames (creator/admin only)
- Promote or demote members to/from admin (creator/admin only)
- Remove members (creator/admin only)
- Leave the workspace (non-creators)
- Delete the workspace (creator only — cascades to all channels and messages)

9.5 Channel Management

Click “Channel settings” at the bottom of the channel sidebar to:

- View channel members
- Add members from the workspace (channel creator only)
- Leave the channel
- Delete the channel (channel creator only)

10 Session Logs

Below are representative session logs demonstrating the core functionality of the system. Each log shows the API calls made and the system's responses.

10.1 Session 1: Registration and Workspace Setup

1. User opens the registration page and creates an account:
 - `register(email="jd1234@nyu.edu", username="janedoe", nickname="Jane", password="pass123")`
 - Response: `{uid: 9, email: "jd1234@nyu.edu", username: "janedoe"}`
2. User creates a new workspace:
 - `create_workspace(wsname="Study Group", wsdescription="Finals prep")`
 - Response: `{wsid: 4, wsname: "Study Group"}`
3. User creates a public channel in the workspace:
 - `create_channel(wsid=4, chname="algorithms", chtype="public")`
 - Response: `{chid: 12, chname: "algorithms", chtype: "public"}`
4. User sends a message:
 - `send_message(chid=12, content="Who is ready for the algorithms final?")`
 - Response: `{msgid: 64, postat: "2026-04-24T10:30:00"}`

10.2 Session 2: Invitations and Collaboration

1. Alice logs in and checks her inbox:
 - `login(username="alicezhang", password="alice123")`
 - `get_invitations() ⇒ [{type: "workspace", id: 2, name: "Tandon Robotics Club"}]`
2. Alice accepts the workspace invitation:
 - `respond_workspace_invite(wsid=2, accept=true)`
 - Alice can now see channels in the Robotics Club workspace.
3. Alice searches for messages containing “robotics”:
 - `search_messages(keyword="robotics") ⇒ Returns results from #events in WS2 (now accessible after accepting the invite).`

10.3 Session 3: Admin Operations

1. Bob logs in and manages the CS Department workspace:
 - `login(username="bobsmith", password="bob456")`
 - `get_workspace_members(wsid=1)` $\Rightarrow$ Lists all members with roles.
2. Bob invites Eve (who has a pending invite) — no duplicate is created due to ON CONFLICT DO NOTHING:
 - `invite_to_workspace(wsid=1, invitee_uid=5)` $\Rightarrow$ `{ok: true}`
3. Bob promotes Carol to admin:
 - `update_workspace_member(wsid=1, target_uid=3, role="admin")` $\Rightarrow$ `{ok: true}`

11 Design Decisions and Revisions for Project 2

11.1 Schema Revisions

The schema from Project 1 was used without structural changes. The `ON DELETE CASCADE` constraints on foreign keys were already in place, and the `status` field on membership tables naturally supports the invitation workflow required by the web interface.

11.2 Design Decisions

- **RPC over REST.** A single endpoint simplifies both the backend (one URL pattern, one dispatcher) and the frontend (one `apiCall` helper). Adding a new feature requires only writing a new Python function and decorating it with `@rpc`.
- **Raw SQL only.** While Django provides a powerful ORM, we exclusively use raw SQL with parameterized queries to satisfy the course requirement and to maintain full control over the generated queries.
- **Session-based authentication over tokens.** Django’s built-in session framework provides secure, server-side session management with minimal configuration. The session cookie is HTTP-only and signed, preventing client-side JavaScript from reading the session ID.
- **Application-level cascade deletes.** Although PostgreSQL supports `ON DELETE CASCADE`, the `delete_workspace` and `delete_channel` functions explicitly delete dependent rows in order. This gives us full control over the deletion order and allows us to wrap the entire operation in a transaction.
- **Docker Compose for deployment.** The entire system (Django + PostgreSQL) runs with a single `docker compose up --build` command, making it easy to set up for the demo.

11.3 Known Limitations

- No WebSocket support — the frontend does not receive real-time updates. Users must refresh or re-select a channel to see new messages.
- No message editing or deletion.
- No file or image uploads.
- Direct channel names are auto-generated as `DM - username` in the frontend but stored as `NULL` in the database.

12 Conclusion

We have designed and implemented *snicker*, a fully functional web-based collaboration system backed by a PostgreSQL relational database. The system supports user registration, workspace and channel management, role-based access control, invitation workflows, messaging, and keyword search — all through a clean React interface communicating with a Django backend via a single RPC endpoint.

Security measures include parameterized queries against SQL injection, React’s automatic XSS escaping, session-based authentication, and transactional consistency for multi-step operations. The system can be deployed locally via Docker Compose for a live demonstration.